# STL Tradeoffs

Container and algorithm choices, with measured numbers


- How this is organised
- Part 1 — Sequence Containers
  - 1.1 `vector` vs `list` — and why `list` is almost never right
  - 1.2 `vector` vs `deque`
  - 1.3 `reserve` — worth it, measured properly
  - 1.4 Growth factor
- Part 2 — Associative Containers
  - 2.1 `map` vs `unordered_map`
  - 2.2 The rehash rules — know these precisely
  - 2.3 `operator[]` inserts
  - 2.4 Small-N: when a `vector` beats a map
  - 2.5 Transparent comparators
  - 2.6 `set` vs sorted `vector`
  - 2.7 What the standard containers don't give you
- Part 3 — Algorithms
  - 3.1 Sorting: pick the weakest one that works
  - 3.2 Erase-remove, and what replaced it
  - 3.3 `find` vs `binary_search` vs member `find`
  - 3.4 `reserve` before a transform
  - 3.5 Parallel algorithms
- Part 4 — Iterators and Invalidation
  - 4.1 The table to know
  - 4.2 The erase-in-a-loop idiom
  - 4.3 Iterator categories
- Part 5 — Strings
  - 5.1 Small string optimisation
  - 5.2 `string_view`
  - 5.3 Concatenation
- Part 6 — Interview Q&A
- The three things worth remembering


# How this is organised

Every section is a **choice** you'd actually have to defend, not a feature list. Numbers were measured on this machine with GCC 13 at `-O2`; treat the **ratios** as the finding, not the absolutes.

The recurring theme: **big-O hides constants, and on modern hardware the constant is usually cache behaviour.** An O(log n) structure that pointer-chases frequently loses to an O(n) scan that streams.

# Part 1 — Sequence Containers

## 1.1 `vector` vs `list` — and why `list` is almost never right

Same work, traversing 200,000 ints and summing:

| Container | Time |
|---|---|
| `vector` | **0.087 ms** |
| `list` | 0.414 ms (**4.8×** slower) |
| `set` | 19.4 ms (**223×** slower) |

Identical operation count. The difference is entirely memory layout.

**Why:** `vector` is contiguous, so the hardware prefetcher sees a linear pattern and stays ahead of you. `list` is a pointer chase — every element is a separate allocation, and the CPU can't prefetch because it doesn't know the next address until it has loaded the current node.

`list` also stores two pointers per element, so an `int` list uses **24 bytes per element** instead of 4 — six times the memory, six times the cache pressure.

### When `list` is actually justified

| Reason | Real? |
|---|---|
| O(1) insert in the middle | Only if you **already have the iterator**. Finding it is O(n), and the scan costs more than the shift would. |
| **References stay valid** across insert/erase | **Yes — this is the real one.** |
| O(1) `splice` between lists | Yes, genuinely unique. |

**And the version that's usually right instead:** an **intrusive** list, where the node pointers live inside your object. No separate allocation per element, one fewer indirection. That's what a matching engine uses for orders within a price level — you need stable references and O(1) unlink, but you don't want an allocation per insertion.

## 1.2 `vector` vs `deque`

Inserting 100,000 elements at the **front**:

| | Time |
|---|---|
| `deque::push_front` | **0.254 ms** |
| `list::push_front` | 2.867 ms |
| `vector::insert(begin())` | **11.4 ms** — and that's only 20,000 elements |

`vector::insert` at the front is O(n) per insert, so $O(n^2)$ overall. That's the same bug as `erase(begin())` in a ring buffer.

`deque` **is a segmented array** — an array of fixed-size blocks. So:

- O(1) push/pop at **both** ends
- O(1) random access, but with an **extra indirection** (find the block, then the offset)
- References stay valid on push at either end (but **not** iterators)
- Traversal is slower than `vector` because of the block boundaries

**Choose** `deque` **when** you need efficient push/pop at both ends — a sliding window, a queue. **Choose** `vector` for everything else.

## 1.3 `reserve` — worth it, measured properly

| | 100k ints | 1M ints |
|---|---|---|
| `push_back` only | 0.453 ms | 5.236 ms |
| `reserve` + `push_back` | **0.221 ms** | **2.320 ms** |

Consistently **~2×**. With `std::string` elements (non-trivial, so reallocation must move them) the gap is similar.

**Why:** 1M `push_back`s caused **21 reallocations**. Capacity doubles each time, so growth is amortised O(1) — but each reallocation copies or moves everything and invalidates all iterators.

**The subtlety:** `reserve` doesn't just save time. It makes iterator invalidation **impossible** within the reserved range, which turns a whole class of dangling-reference bug into a non-possibility. In a hot path that matters more than the 2×.

### 1.4 Growth factor

libstdc++ and libc++ grow by **2×**; MSVC uses **1.5×**.

The argument for 1.5: with doubling, the newly allocated block is always larger than the sum of all previously freed blocks, so freed memory can never be reused for the next growth. 1.5 allows reuse after a few growths. The counter-argument is fewer reallocations with 2×.

**You mostly don't care** — you should be reserving. But it's a good "do you know your library" question.

---

## Part 2 — Associative Containers

### 2.1 `map` vs `unordered_map`

200,000 inserts:

| | Time |
|---|---|
| `std::map` (red-black tree) | 79.6 ms |
| `std::unordered_map` | 31.7 ms |
| `unordered_map` + `reserve` | **19.9 ms** |

`reserve` gives another **1.6×** by eliminating rehashing.

| | `map` | `unordered_map` |
|---|---|---|
| Structure | red-black tree | bucket array + node chains |
| Lookup | O(log n) | O(1) average, **O(n) worst case** |
| Ordered iteration | **yes** | no |
| `lower_bound` / range queries | **yes** | no |
| Iterator invalidation on insert | none | **on rehash** |
| Reference invalidation on insert | none | **none** (node-based) |
| Memory per element | ~3 pointers + colour | node + bucket slot |

**Choose `map` when** you need ordered iteration or range queries — a price level book where you walk from the touch outward.

**Choose `unordered_map` when** you only need key lookup — and **always** `reserve` if you know the size.

### 2.2 The rehash rules — know these precisely

Rehash happens when `size + 1 > max_load_factor × bucket_count` (default load factor 1.0). It allocates a bigger bucket array and **relinks every node**.

| | Survives a rehash? |
|---|---|
| **References** to mapped values | **Yes** |
| **Pointers** to elements | **Yes** |
| **Iterators** | **No** |
| A **traversal in progress** | **No** |

References survive because it's **node-based** — each element has its own allocation that never moves. Only the bucket pointer array is rebuilt.

**This distinction is exactly why a concurrent rehash bug is so subtle.** Holding `auto& v = m[key]` is safe across a rehash, so the code looks fine. What races is the **bucket-chain walk to reach it** — computing `hash % bucket_count` against a changing count, indexing a freed array, following `->next` pointers mid-relink. There's no torn *value*; there's a torn *journey*.

## 2.3 `operator[]` inserts

```
m[key]                // default-constructs and INSERTS if absent
m.at(key)             // throws if absent
m.find(key)           // returns end() if absent
m.contains(key)       // C++20
m.try_emplace(k, args) // constructs only if absent -- no wasted temporary
```

**`operator[]` cannot be `const`**, precisely because it may modify the container. If your read path compiles with `operator[]`, the method isn't `const`, and that's the signal.

Use `find` on read paths, always.

## 2.4 Small-N: when a `vector` beats a map

Lookup by key, 200,000 repetitions:

| n | `vector` linear scan | `map` | `unordered_map` |
|---|---|---|---|
| 8 | **0.77 ms** | 0.66 ms | 0.97 ms |
| 32 | 1.31 ms | 1.02 ms | **0.98 ms** |
| 128 | 6.70 ms | 1.38 ms | **0.98 ms** |
| 512 | 19.53 ms | 1.79 ms | **1.01 ms** |

**The honest reading:** the "linear scan always wins for small n" claim is **overstated**. Here it's only competitive up to about n=8–16, and `unordered_map` is flat and hard to beat from n≈32 upward.

It depends heavily on element size and comparison cost — for a `vector` of small POD with an integer compare the crossover is higher; for anything requiring a string compare it's lower.

**The point to make in an interview isn't "linear scan wins" — it's "measure the crossover for your element type, don't assume."**

## 2.5 Transparent comparators

```
struct SvHash {
    using is_transparent = void;
    size_t operator()(std::string_view s) const { return std::hash<std::string_view>{}(s); }
};
struct SvEq {
    using is_transparent = void;
    bool operator()(std::string_view a, std::string_view b) const { return a == b; }
};
std::unordered_map<std::string, V, SvHash, SvEq> m;
m.find(some_string_view);      // no std::string constructed
```

Without it, `find(string_view)` **constructs a temporary `std::string`** — a heap allocation per lookup for anything past the small-string buffer.

**Two gotchas:**

1. You need `is_transparent` on **both** the hash and the equality functor, or it silently falls back to constructing the key.
2. **Small string optimisation** means strings up to ~15 characters live inline and don't allocate at all. So this only pays for longer keys — an NSE option symbol like `NIFTY25SEP24000CE` is 17 characters, so it's just past the threshold. Below 15, transparent lookup buys you very little.

### 2.6 `set` vs sorted `vector`

For a **read-mostly** set, a sorted `vector` with `std::binary_search` usually wins outright: contiguous, cache-friendly, half the memory, and O(log n) lookup just like `set`.

`set` wins when insertions and deletions are frequent and interleaved with lookups, since a sorted vector insert is O(n).

The traversal number from §1.1 makes the case: summing 200k elements took 0.087 ms from a `vector` and **19.4 ms** from a `set` — a 223× difference for the same arithmetic.

### 2.7 What the standard containers don't give you

`unordered_map` is **mandated** to be node-based with bucket chaining. That means a pointer chase per lookup, guaranteed, forever — the standard rules out an open-addressing implementation.

That's why performance-sensitive codebases use flat hash maps — `absl::flat_hash_map`, `robin_hood`, `ankerl::unordered_dense`. They store elements **inline** in one array, so a lookup is typically one cache miss instead of two or three.

**The tradeoff:** flat maps invalidate **references** on rehash, not just iterators, because elements move. So you can't hold a reference across an insert. That's a real constraint, and it's worth naming — it's the reason the standard chose node-based.

## Part 3 — Algorithms

### 3.1 Sorting: pick the weakest one that works

500 elements, 20,000 repetitions, cycling 256 datasets so the branch predictor can't memorise:

| Algorithm | Time | vs full sort |
|---|---|---|
| `std::sort` (full order) | 341.6 ms | 1.00× |
| `std::stable_sort` (full order) | 383.9 ms | 0.89× |
| **`partial_sort`** (top 20) | **66.5 ms** | **5.1×** |
| `nth_element` (top 20, unordered) | 75.8 ms | 4.5× |

**`partial_sort` is 5× faster than a full sort** when you only want the extremes.

| Algorithm | Complexity | Gives you |
|---|---|---|
| `sort` | O(n log n) | full order, **not stable** |
| `stable_sort` | O(n log n) | full order, stable, **needs extra memory** |
| `partial_sort` | O(n log k) | first k in order, rest unspecified |
| `nth_element` | **O(n)** | kth element correct, rest partitioned |

**Surprising result above:** `nth_element` is O(n) and `partial_sort` is O(n log k), yet `partial_sort` measured *faster* here. At k=20, `log k` is about 4.3 — small enough that constants dominate, and `nth_element`'s introselect does more work per element. **Asymptotic complexity doesn't decide anything at n=500.**

#### `std::sort` internals

Introsort: quicksort, switching to heapsort at depth `2·log n` (guaranteeing O(n log n) worst case), plus insertion sort for small ranges.

**It is not stable**, and this is a real correctness issue, not a curiosity. In a ranking engine, equal values come out in an order determined by pivoting — so the same input ranks differently between runs, and you can never reconcile live output against a backtest. Fix by making the comparator a **total order** with a tiebreak on a stable id.

#### Strict weak ordering

Your comparator must be irreflexive, antisymmetric, transitive, and have transitive equivalence. **Violating it is undefined behaviour** in `std::sort`, not merely a wrong order — some implementations read past the end of the array.

NaN violates it: `a < NaN` and `NaN < a` are both false so they look "equivalent", but two different values both "equivalent" to NaN aren't equivalent to each other.

Measured effect: 72 NaNs among 500 elements left **25 perfectly valid entries out of order**.

### 3.2 Erase-remove, and what replaced it

```
// pre-C++20: remove shuffles survivors forward and returns the new logical end
v.erase(std::remove(v.begin(), v.end(), value), v.end());

// C++20
std::erase(v, value);
std::erase_if(v, pred);
```

`std::remove` **does not remove anything** — it can't, algorithms don't know about containers. It returns an iterator to the new end, and you must erase the tail yourself. Forgetting the second half leaves garbage in the container with the size unchanged.

### 3.3 `find` vs `binary_search` vs member `find`

```
std::find(v.begin(), v.end(), x)     // O(n) linear
std::binary_search(v.begin(), v.end(), x)   // O(log n), REQUIRES sorted
s.find(x)                            // member: O(log n) for set, O(1) for unordered_set
```

**Always prefer the member `find` when it exists** — `std::find` on a `set` is O(n) and ignores the structure entirely. This is a common accidental pessimisation.

### 3.4 `reserve` before a transform

```
std::vector<Out> out;
out.reserve(in.size());                      // <-- the important line
std::transform(in.begin(), in.end(), std::back_inserter(out), f);
```

`back_inserter` calls `push_back`, so without the reserve you pay the reallocation sequence again.

### 3.5 Parallel algorithms

```
#include <execution>
std::sort(std::execution::par, v.begin(), v.end());
```

**Two warnings worth raising:**

1. **libstdc++ requires TBB to be linked.** Without it, it **silently runs serially** — a performance bug with no diagnostic at all.
2. `par_unseq` forbids allocation, locks, and any synchronisation in the body. Violating it is UB.

And the general point: parallelism only pays when the work per element is large enough to amortise scheduling. Thread creation and join cost 14–25 µs. If the whole job is 300 ns, you'll measure **46–82× slower** — which I did.

---

# Part 4 — Iterators and Invalidation

### 4.1 The table to know

| Container | Operation | Iterators | References |
|---|---|---|---|
| `vector` | `push_back` (realloc) | **all invalid** | **all invalid** |
| `vector` | `push_back` (no realloc) | end() only | valid |
| `vector` | `erase` | from erase point | from erase point |
| `deque` | `push_front` / `push_back` | **all invalid** | **valid** |
| `deque` | `erase` in middle | all invalid | all invalid |
| `list` | any insert/erase | only the erased one | only the erased one |
| `map` / `set` | insert | **valid** | **valid** |
| `map` / `set` | erase | only the erased one | only the erased one |
| `unordered_map` | insert **with rehash** | **all invalid** | **valid** |
| `unordered_map` | insert without rehash | valid | valid |

**The two rows people get wrong:** `deque::push_front` invalidates iterators but **not** references; `unordered_map` rehash invalidates iterators but **not** references.

### 4.2 The erase-in-a-loop idiom

```
for (auto it = m.begin(); it != m.end(); ) {
    if (pred(*it)) it = m.erase(it);      // erase returns the next iterator
    else           ++it;
}
```

`it++` inside the loop after erasing is a use-after-free. The `erase`-returns-next form is the safe one.

### 4.3 Iterator categories

Input → Forward → Bidirectional → Random Access → Contiguous (C++20).

**Why it matters practically:** `std::sort` needs random access, so **you cannot sort a `list`** — it has its own `list::sort` member. And `std::distance` is O(1) on random access, O(n) otherwise, so calling it in a loop on a `list` is accidentally quadratic.

---

## Part 5 — Strings

### 5.1 Small string optimisation

libstdc++ stores strings up to **15 characters** inline in the object (`sizeof(std::string)` is 32 bytes), avoiding a heap allocation entirely. Longer strings allocate.

**Practical consequence:** an NSE option symbol like `NIFTY25SEP24000CE` is 17 characters — just past the threshold. So every construction of it is a heap allocation, which is exactly why transparent lookup by `string_view` earns its keep for symbols and buys almost nothing for short tickers.

### 5.2 `string_view`

A non-owning pointer + length. Free to construct, free to substring.

**The trap:** it does not own the data. This dangles:

```
std::string_view sv = std::string("temp") + "orary";   // temporary dies at the semicolon
```

**And it is not NUL-terminated.** Passing `sv.data()` to a C API that expects a C string is a bug.

### 5.3 Concatenation

```
s = a + b + c + d;                    // several temporaries
s.reserve(a.size()+b.size()+...);     // then append -- one allocation
s.append(a).append(b);
```

For heavy formatting, `std::format` (C++20) or `fmt` beats both `ostringstream` and manual concatenation.

---

# Part 6 — Interview Q&A

**"Why is `list` almost never the right container?"** Cache locality. Same traversal of 200k ints: 0.087 ms for `vector`, 0.414 ms for `list` — nearly 5× for identical work, because a linked list is a pointer chase the prefetcher can't follow. It also costs 24 bytes per `int`. The only genuine reasons are stable references across insertion and O(1) `splice` — and if you need stable references, an intrusive list is usually better still.

**"`map` or `unordered_map`?"** `map` if you need ordered iteration or range queries; `unordered_map` otherwise, and always `reserve`. Measured on 200k inserts: 79.6 ms vs 31.7 ms, dropping to 19.9 ms with `reserve`.

**"When does `unordered_map` invalidate things?"** On rehash. **Iterators are invalidated, references are not** — it's node-based, so the elements themselves never move; only the bucket array is rebuilt. That distinction is exactly what makes a concurrent rehash bug subtle, because your reference stays valid and the code looks safe.

**"Is linear search faster than a hash map for small n?"** It's overstated. I measured the crossover at around n=8–16 for ints; `unordered_map` was already winning at n=32 and flat thereafter. It depends on element size and comparison cost. The real answer is measure the crossover for your type rather than assuming either way.

**"When would you use `nth_element`?"** When you need the kth element or a partition but not a full order — it's O(n) versus O(n log n). Though at n=500 wanting the top 20, `partial_sort` measured *faster* despite worse asymptotics, because constants dominate at that size. That's the more useful lesson.

**"Is `std::sort` stable? Does it matter?"** No, and it does. In a ranking engine, equal values come out in pivot-dependent order, so the same input ranks differently between the batch and daily runs and you can never reconcile against a backtest. Fix with a total order — value, then a stable id tiebreak.

**"What happens if your comparator isn't a strict weak ordering?"** Undefined behaviour, not just a bad order — some implementations read past the end. NaN is the common cause, because it compares false against everything, so transitivity of equivalence fails.

**"Why does `std::remove` not remove?"** Algorithms operate on iterators and don't know about containers, so they can't change the size. It shuffles survivors forward and returns the new logical end; you erase the tail yourself. C++20's `std::erase` does both.

**"What's `emplace_back` vs `push_back`?"** `emplace_back` forwards arguments to construct in place, avoiding a temporary. With an already-constructed object they're equivalent. One caveat: `emplace_back` uses direct initialisation, so it can invoke `explicit` constructors that `push_back` would reject.

**"How would you make an `unordered_map` faster?"** `reserve` up front to eliminate rehashing (1.6× measured). Transparent comparators to avoid constructing keys on lookup. And if it's genuinely hot, a flat hash map — the standard mandates node chaining, so a pointer chase per lookup is guaranteed. The tradeoff is that flat maps invalidate references on rehash, since elements move.

**"When would you not use the STL at all?"** When you need control it can't give: a fixed-capacity ring buffer with no allocation, an intrusive list where the nodes live inside your objects, or a preallocated object pool. All three appear in a matching engine, and all three exist because the STL's allocation behaviour is wrong for a hot path — not because its algorithms are.

---

# The three things worth remembering

1. **Big-O hides constants, and the constant is usually cache behaviour.** A `vector` scan beating a tree isn't an

anomaly; it’s the normal case at small and medium n.

2. **Pick the weakest algorithm that answers your question.** Full sort when you want the top 20 is 5× more work than necessary — and that was a bigger win than parallelism could have delivered.

3. `reserve` **isn’t only a speed optimisation.** It makes reallocation impossible in the reserved range, which turns a whole class of dangling-reference bug into a non-possibility.